

Explorando *os Widgets*

Até aqui, os agentes respondiam com texto ou JSON. **Widgets** são componentes visuais do ChatKit (cartões, listas, botões, formulários) que aparecem dentro da conversa. Nesta aula, desenhamos no **Widget Builder** a **Trilha de Estudo** do curso, entendemos o que há dentro de um arquivo `.widget`, ligamos o widget ao agente pelo Output format e tratamos **ações**: botões que abrem links e um formulário que só age com confirmação.

Card · ListView

Widget Builder

Arquivo `.widget`

Output format: Widget

Ações e `onAction`

Formulários

Formação Claude Code 2026 · Baseado em developers.openai.com (ChatKit widgets, Actions in ChatKit, Theming), no cookbook "Build, deploy, and optimize agentic workflows with AgentKit" e nos tipos do repositório [openai/chatkit-js](https://github.com/openai/chatkit-js), consultados em setembro de 2026 · Telas: reprodução ilustrativa

Por que *widgets*

Uma lista de cinco aulas com título, módulo, motivo e link fica confusa em texto corrido. O mesmo conteúdo num **cartão com linhas e botões** é lido num relance e permite agir com um clique. Segundo a documentação, widgets são os **contêineres e componentes** do ChatKit: você pode usar os prontos, adaptar modelos ou desenhar os seus.

SAÍDA DO AGENTE	BOM PARA	LIMITE
Texto	Explicações, conversas	Listas e comparações ficam difíceis de ler
JSON (aulas 02 e 05)	Passar dados para outro nó ou para o seu código	Não é feito para o usuário ler
Widget (esta aula)	Mostrar dados estruturados e oferecer ações	Só aparece onde há ChatKit

O ponto-chave: o widget é um **JSON com esquema que vira interface**. Tudo o que você aprendeu sobre saída estruturada continua valendo; muda só quem lê o resultado.

Contêineres *e componentes*

Todo widget tem **um contêiner raiz** (`WidgetRoot`) com vários **componentes** dentro (`WidgetNode`).

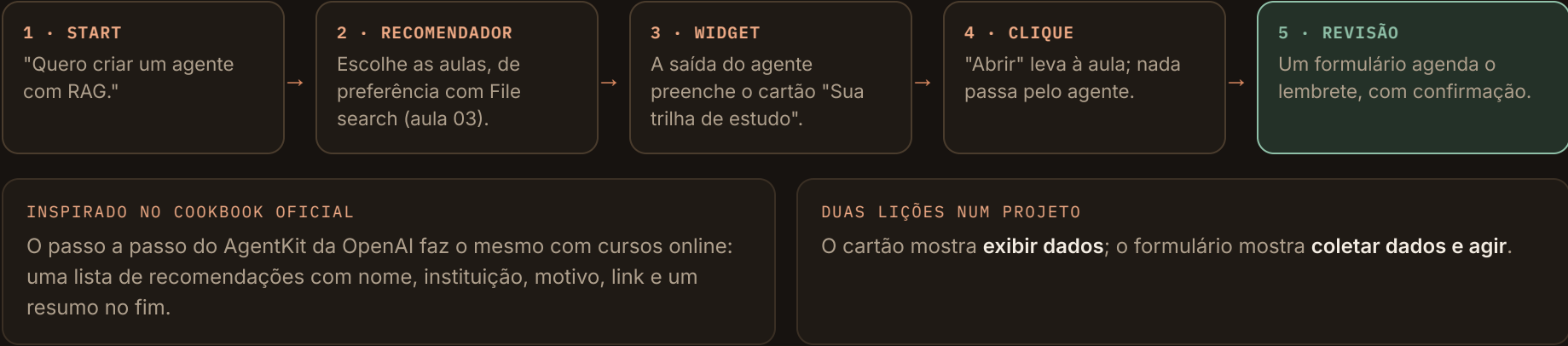
CONTÊINER	CARACTERÍSTICAS
Card	Caixa delimitada; aceita <code>status</code> e os botões <code>confirm</code> e <code>cancel</code> ; tamanhos sm, md, lg e full
ListView	Lista vertical de <code>ListViewItem</code> , cada um clicável
Basic	Contêiner simples, sem moldura

GRUPO	COMPONENTES
Texto	Title, Text, Caption, Markdown, Badge, Label
Layout	Row, Col, Box, Spacer, Divider, Table
Mídia	Image, Icon
Controles	Button, Form, Input, Textarea, Select, Checkbox, RadioGroup, DatePicker

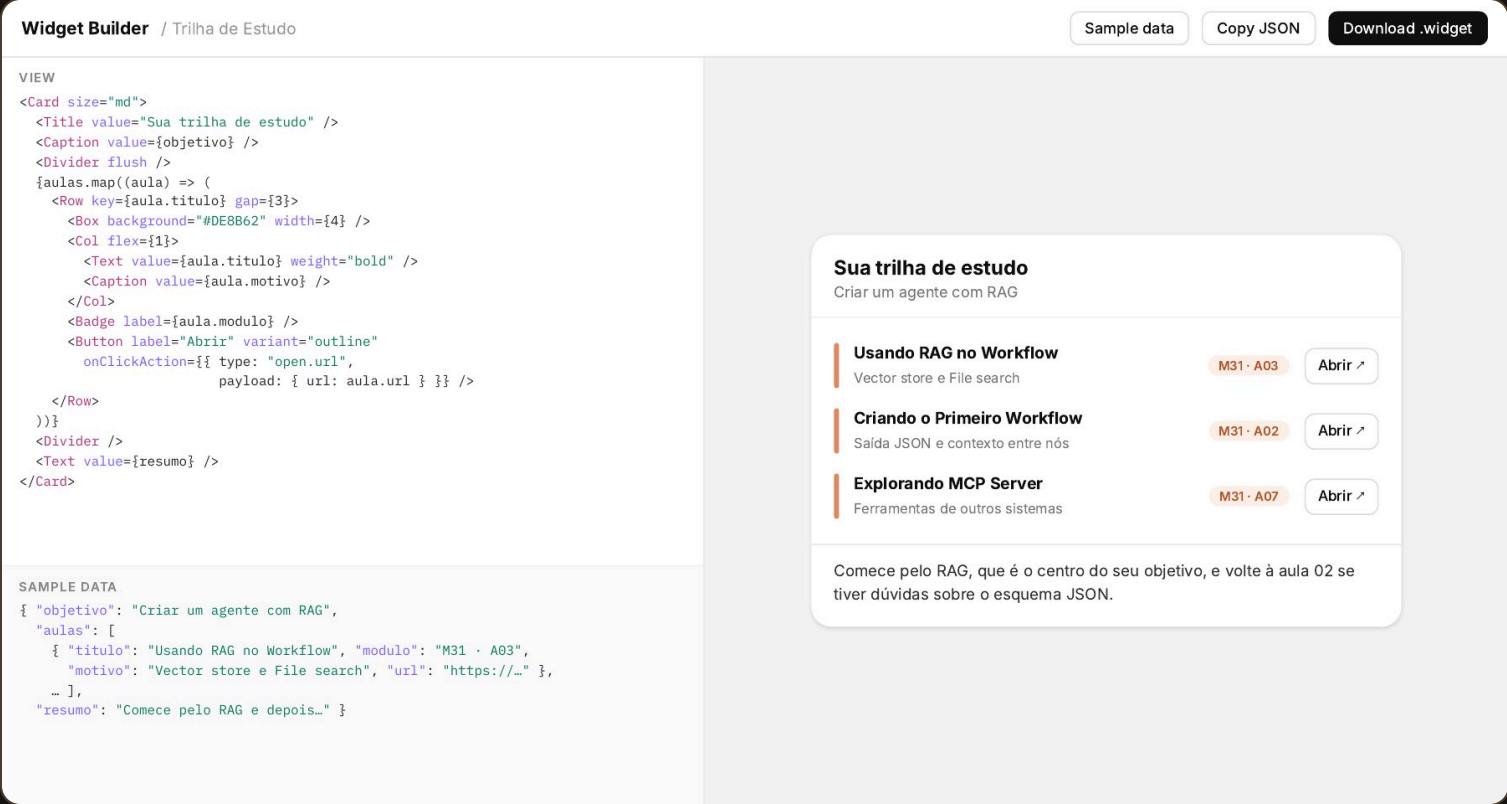
Pense em HTML com peças contadas: não há CSS livre nem script. Isso garante que o widget se ajuste ao tema do ChatKit (claro ou escuro) e seja **seguro de exibir**.

A Trilha de Estudo

O aluno diz o que quer aprender; o agente recomenda de 3 a 5 aulas do curso, na ordem certa, com o motivo de cada uma e um botão para abrir. Depois, oferece agendar uma revisão.



Desenhar no *Widget Builder*



View (código), dados de exemplo e prévia lado a lado - reprodução ilustrativa (a interface é em inglês)

COMECE DESCREVENDO

O Widget Builder gera um **ponto de partida com IA** a partir de uma descrição. Depois você edita.

TRÊS ÁREAS

O **código** do widget, os **dados de exemplo** e a **prévia** de como o usuário vai ver.

FICA PRONTO

Quando gostar, baixe o arquivo **.widget**.

A view, *em JSX*

```
<Card size="md">
  <Title value="Sua trilha de estudo" />
  <Caption value={objetivo} />
  <Divider flush />
  {aulas.map((aula) => (
    <Row key={aula.titulo} gap={3}>
      <Box background="#DE8B62" width={4} />
      <Col flex={1}>
        <Text value={aula.titulo} weight="bold" />
        <Caption value={aula.motivo} />
      </Col>
      <Badge label={aula.modulo} />
      <Button label="Abrir" variant="outline"
        onClickAction={{
          type: "open.url",
          payload: { url: aula.url },
        }} />
    </Row>
  ))}
  <Divider />
  <Text value={resumo} />
</Card>
```

AS CHAVES

`{objetivo}`, `{aulas}` e `{resumo}` são os **dados**. A view só diz *como* mostrá-los; quem os fornece é o agente.

O LAÇO

`aulas.map(...)` repete a linha para cada aula: 3 ou 5, o cartão se ajusta.

O BOTÃO

A ação `open.url` abre o link. É o mesmo tipo de ação usado no exemplo oficial do cookbook.

O esquema *dos dados*

```
{
  "type": "object",
  "properties": {
    "objetivo": { "type": "string" },
    "aulas": {
      "type": "array",
      "minItems": 3,
      "items": {
        "type": "object",
        "properties": {
          "titulo": { "type": "string" },
          "modulo": { "type": "string" },
          "motivo": { "type": "string" },
          "url": { "type": "string", "format": "uri" }
        },
        "required": ["titulo", "modulo", "motivo", "url"],
        "additionalProperties": false
      }
    },
    "resumo": { "type": "string" }
  },
  "required": ["objetivo", "aulas", "resumo"],
  "additionalProperties": false
}
```

Junto da view, o widget carrega um **JSON Schema** dos dados. É ele que o agente vai seguir.

É o mesmo tipo de esquema das aulas 02 e 05. A diferença: em vez de você ler o JSON, o ChatKit o **desenha**.

`minItems: 3` garante uma trilha de verdade; `format: "uri"` pede um link válido. Restrições no esquema evitam cartões vazios ou quebrados.

O que há num *arquivo .widget*

O `.widget` baixado é um JSON. Abrindo o arquivo de exemplo publicado no cookbook da OpenAI, encontramos estes campos:

CAMPO	O QUE GUARDA
<code>version</code> e <code>name</code>	Versão do formato e nome do widget
<code>template</code>	A estrutura do widget em JSON, com marcações de modelo para encaixar os dados (laços e variáveis)
<code>jsonSchema</code>	O esquema dos dados que o agente precisa produzir
<code>outputJsonPreview</code>	O widget já montado com os dados de exemplo
<code>encodedWidget</code>	O projeto do Widget Builder (incluindo a view em JSX), codificado

Não edite o `.widget` à mão: o formato não tem documentação própria. Mude no Widget Builder e **baixe de novo**. Guarde o arquivo no repositório do projeto, como guardaria qualquer fonte.

Ligar o widget *ao agente*

1 No nó Recomendador, em Output format, escolha Widget.

2 Envie o arquivo `trilha-de-estudo.widget`.

3 Ajuste as instruções para dizer **que dados** buscar para cada campo.

Segundo o cookbook, o agente **passa a saber sozinho** o formato JSON que preenche o widget. As instruções servem para dizer de onde vêm os dados e como escolhê-los.

Instructions · Recomendador

Você monta trilhas de estudo da Formação Claude Code 2026.

A partir do objetivo do aluno, recomende de 3 a 5 aulas do curso, na ordem em que devem ser estudadas.

- Use SOMENTE aulas encontradas nos materiais (File search). Nunca invente título, módulo ou link.
- título: nome exato da aula.
- modulo: no formato "M31 · A03".
- motivo: uma frase sobre por que esta aula ajuda no objetivo.
- url: o link oficial da aula.
- resumo: duas frases sobre a ordem sugerida.

O widget *na conversa*

Trilha de Estudo

Preview

Close

quero aprender a criar um agente com RAG. por onde começo?

Sua trilha de estudo

Criar um agente com RAG

Usando RAG no Workflow

Vector store e File search

M31 · A03

Abrir ↗

Criando o Primeiro Workflow

Saída JSON e contexto entre nós

M31 · A02

Abrir ↗

Explorando MCP Server

Ferramentas de outros sistemas

M31 · A07

Abrir ↗

Comece pelo RAG, que é o centro do seu objetivo, e volte à aula 02 se tiver dúvidas sobre o esquema JSON.

Agendar revisão

Receba um lembrete para revisar a trilha

Data

15/10/2026

Canal

E-mail

Agora não

Agendar

Nada é agendado até você confirmar.

Message...

O cartão da trilha e o formulário de revisão · reprodução ilustrativa

CONFIRA OS DADOS

Cada título e link existe no curso? Um widget bonito com dado inventado engana mais que texto.

TESTE OS EXTREMOS

Objetivo vago, objetivo fora do curso, 5 aulas com títulos longos.

TESTE OS DOIS TEMAS

O ChatKit pode estar claro ou escuro: cores fixas como `#DE8B62` precisam funcionar nos dois.

Quando o usuário *clica*

Componentes interativos recebem uma **ação** (`ActionConfig`): um `type` , um `payload` e, opcionalmente, quem a trata (`handler`).

AÇÃO	QUEM TRATA	EXEMPLO
<code>open.url</code>	O próprio ChatKit	O botão "Abrir" da trilha
Ação personalizada com <code>handler: "client"</code>	O seu front-end, no callback <code>widgets.onAction</code>	"Agendar" chama a API do seu site
Ação personalizada no servidor	O seu servidor ChatKit (ChatKit Python SDK), que pode atualizar o widget ou responder na conversa	Salvar a revisão e responder "Agendado!"

Com o workflow **hospedado** pelo Agent Builder, o caminho mais direto para ações próprias é o `onAction` no front-end, que repassa ao seu back-end, como no exemplo da documentação.

Compare com a aula 06: na client tool, o **agente** pede algo ao navegador; na ação de widget, quem dispara é o **usuário**, com um clique.

Tratar a ação *no front-end*

```
chatkit.setOptions({
  api: { getClientSecret },
  widgets: {
    async onAction(action, item) {
      if (action.type !== "agendar_revisao") return;

      // payload traz os campos do formulário, pelo name
      const { data, canal } = action.payload;

      const res = await fetch("/api/revisoes", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ data, canal, itemId: item.id }),
      });

      // avisa o servidor do ChatKit, se você tiver um
      await chatkit.sendAction(
        { type: "revisao_agendada", payload: { ok: res.ok } },
        item.id,
      );
    },
  },
});
```

A ASSINATURA

Pelos tipos do `chatkit-js`, o callback recebe a **ação** e o **item do widget** que a originou. `sendAction` envia uma ação ao servidor, opcionalmente ligada a esse item.

O `payload` vem do navegador, como o resultado de uma client tool: o seu back-end **valida tudo de novo** (data no futuro, canal permitido, usuário logado).

Coletar dados *com confirmação*

```
<Card size="sm" asForm
  confirm={{ label: "Agendar",
    action: { type: "agendar_revisao",
      handler: "client" } }}
  cancel={{ label: "Agora não",
    action: { type: "cancelar_revisao",
      handler: "client" } }}>
  <Title value="Agendar revisão" />
  <Label value="Data" fieldName="data" />
  <DatePicker name="data" />
  <Label value="Canal" fieldName="canal" />
  <Select name="canal" options={{[
    { value: "email", label: "E-mail" },
    { value: "whatsapp", label: "WhatsApp" },
  ]}} />
</Card>
```

ASFORM

Com `asForm`, o `Card` se comporta como um `Form`: **valida** os campos e envia os valores para a ação de `confirm`.

O NAME VIRA CHAVE

Cada campo entra no `payload` pelo seu `name`: `data` vira `action.payload.data`. Nomes repetidos colidem; use nomes únicos.

Nada acontece até o clique em **Agendar**: é o "ação visível pede um sim" das aulas 04 e 06, agora com formulário.

Widgets que *ajudam*

- ✓ Um widget, uma ideia: trilha num cartão, agendamento em outro
- ✓ Poucos itens por lista: `ListView` tem `limit`
- ✓ Textos curtos: o widget não é lugar para parágrafos
- ✓ Botões com verbo claro: "Abrir", "Agendar", "Agora não"
- ✓ Cores que funcionam nos temas **claro e escuro**
- ✓ `alt` em toda `Image` (acessibilidade)
- ✓ Um **texto curto** fora do widget, para quem lê por notificação ou leitor de tela
- ✓ Versão do `.widget` guardada junto do projeto

Links vêm do modelo. Uma URL num botão `open.url` pode ter sido inventada ou injetada. Instrua o agente a usar só links das fontes e, se puder, confira o domínio (aula 07).

Nada sensível no payload. Tokens e dados pessoais não vão em ações: o payload circula pelo navegador e pela conversa.

Widgets *sem o Agent Builder*

```
# No seu servidor ChatKit (ChatKit Python SDK)
from chatkit.widgets import Card, Title, Text, Button
from chatkit.actions import ActionConfig

def cartao_trilha(trilha) -> Card:
    linhas = [
        Button(
            label=f"Abrir: {a.titulo}",
            variant="outline",
            onClickAction=ActionConfig(
                type="open.url", payload={"url": a.url}
            ),
        )
        for a in trilha.aulas
    ]
    return Card(children=[
        Title(value="Sua trilha de estudo"),
        *linhas,
        Text(value=trilha.resumo),
    ])

# trilha = saída estruturada de um Agent do Agents SDK
# (output_type com objetivo, aulas e resumo, como na página 06)
```

O QUE CONTINUA E O QUE MUDA

- O **ChatKit** continua disponível: só o Agent Builder sai do ar em 30/11/2026
- O agente vira um **Agent** do SDK com **output_type** igual ao **jsonSchema** do widget
- O **seu servidor** monta o widget com os componentes do ChatKit Python SDK e o envia à conversa
- Ações podem ser tratadas no servidor, além do **onAction** no cliente

O **.widget** continua útil como **especificação**: o esquema e a prévia dizem exatamente o que construir. Confira os nomes exatos dos componentes no guia do ChatKit Python SDK.

Erros comuns *com widgets*

Widget bonito com dado inventado. O visual passa confiança; a regra "só das fontes" é ainda mais importante.

Confiar no payload. Ele vem do navegador: valide no back-end, sempre.

Editar o .widget à mão. Formato sem documentação; mude no Widget Builder e baixe de novo.

Ação sem confirmação. Agendar, comprar e enviar pedem um botão de confirmar.

Testar só no tema claro. Cores fixas somem no escuro.

Esperar widget pela API pura. Widget é do ChatKit; sem ele, sobra só o JSON.

Teste o que *ficou*

1. Como o agente passa a responder com o widget?

- A) Colando o JSX nas instruções
- B) Output format "Widget" com o arquivo .widget enviado
- C) Com uma client tool
- D) Pelo nó If/else

B. O agente segue o esquema do widget.

2. O que o `jsonSchema` do widget define?

- A) As cores do cartão
- B) Os dados que o agente precisa produzir para preencher o widget
- C) O tema do ChatKit
- D) As ações permitidas

B. A view mostra; o esquema diz o que mostrar.

3. Em um `Card` com `asForm`, onde chega o valor do campo `name="data"` ?

- A) Nas instruções do agente
- B) Em `action.payload.data` da ação de confirm
- C) Num vector store
- D) Em lugar nenhum

B. O name vira chave do payload.

4. O que acontece com os widgets em 30/11/2026?

- A) Deixam de existir
- B) Continuam no ChatKit; muda quem gera os dados e monta o widget
- C) Viram imagens
- D) Só funcionam no Preview

B. Só o Agent Builder sai do ar.

A aula *em uma página*

Widgets transformam a saída estruturada em interface: um contêiner (Card, ListView) com componentes de texto, layout e controle. Desenhamos a Trilha de Estudo no Widget Builder, vimos que o `.widget` guarda **template, esquema e prévia**, ligamos o arquivo ao agente pelo Output format e escrevemos instruções que dizem de onde vem cada campo.

Nas ações, `open.url` abre links; ações próprias chegam ao `onAction`; e um `Card` com `asForm` coleta dados e só age com confirmação, sempre com validação no back-end.

PEÇA	PAPEL
View (JSX)	Como mostrar
<code>jsonSchema</code>	O que o agente produz
Output format: Widget	Liga o widget ao nó
<code>open.url</code>	Abre link, sem código
<code>widgets.onAction</code>	Ações próprias no cliente
<code>asForm</code> + confirm	Formulário com confirmação

Do canvas ao produto, *e além de 30/11*

Em oito aulas, saímos do canvas em branco e chegamos a um agente completo: workflows com saída estruturada, RAG com File search, os templates Customer service e Data enrichment, client tools no navegador, servidores MCP e widgets no ChatKit. Em cada aula vimos o mesmo padrão no **Agents SDK**, que é para onde os seus workflows vão quando o Agent Builder sair do ar.

01 · Conhecendo

02 · Primeiro workflow

03 · RAG

04-05 · Templates

06 · Client tool

07 · MCP

08 · Widgets

Próximo passo → exportar seus workflows (Code → Agents SDK) e testá-los antes de 30/11/2026

Fontes: developers.openai.com (ChatKit widgets, Actions in ChatKit, Theming, Migrate from Agent Builder e Deprecations); cookbook "Build, deploy, and optimize agentic workflows with AgentKit" e o arquivo .widget de exemplo publicado nele; tipos do pacote em github.com/openai/chatkit-js, consultados em setembro de 2026. As telas são reproduções ilustrativas feitas para o curso.